

CORBA - *Common Object Request Broker Architecture*

Bruno Fin Ferreira, Felipe Augusto Kuentzer, Henrique Alvares Chamorra, Odorico Mendizabal
PUCRS - Pontifícia Universidade Católica do Rio Grande do Sul
Av. Ipiranga, 6681 Prédio 32 sala 507, Porto Alegre, Brasil
E-mail: {bruno.ferreira,felipe.kuentzer, henrique.chamorra,odorico.mendizabal}@acad.pucrs.br

Resumo — Aplicações para sistemas distribuídos permitem que nodos remotos interajam entre si de forma colaborativa, possibilitando aumento no desempenho, disponibilidade e integração entre sistemas já existentes. Entretanto, características inerentes aos ambientes em que tais aplicações executam, como a ausência de um espaço de endereçamento comum e a distribuição física entre os participantes, requerem mecanismos de comunicação entre processos. Este artigo apresenta o padrão CORBA, que descreve uma arquitetura distribuída em que objetos podem ser instanciados remotamente. CORBA faz uso das características do paradigma orientado a objetos para proporcionar uma semântica familiar a maioria dos programadores. Dessa forma, é oferecido um nível de abstração mais alto daquele oferecido por sockets, permitindo a transparência de localização, concorrência e, em alguns casos, transparência de falhas.

Palavras chaves — CORBA, Sistemas Distribuídos, Objetos Remotos.

I. INTRODUÇÃO

O desenvolvimento de aplicações para sistemas distribuídos sempre foi complexo, devido principalmente as restrições impostas pelo próprio ambiente distribuído e heterogeneidade de plataformas. A ocorrência de falhas, diferenças significativas na latência de rede e qualidade nos links de conexão dos processos distribuídos ocasionam uma série de incertezas sobre a entrega de mensagens.

Além disso, aplicações executadas em diferentes arquiteturas de hardware e/ou desenvolvidas em diferentes linguagens de programação também costumam apresentar diferenças em como lidam com codificação de dados e representação de suas estruturas internas.

Visando facilitar o desenvolvimento dessas aplicações, surgiram diferentes *middlewares* que oferecem maior abstração ao desenvolvedor de aplicações distribuídas. Dessa forma, aspectos como transparência de localização, detecção de falhas e heterogeneidade passam a ser gerenciados em um nível intermediário, sem intervenção do desenvolvedor.

Com o intuito de prover um mecanismo de gerenciamento de objetos distribuídos, surge o CORBA (*Common Object Request Broker Architecture*), um *middleware* que abstrai parte da comunicação em sistemas distribuídos, oferecendo ao programador um modelo de invocação a objetos remotos. Assim, durante o desenvolvimento das aplicações, o programador não precisa tomar conhecimento da localização, arquitetura do hardware e linguagem de programação utilizada por outros clientes ou servidores. Outros serviços

especializados também são oferecidos pelo CORBA, como a geração e notificação de eventos remotos.

Este artigo está organizado como segue. A próxima seção descreve o CORBA. Em seguida, a Seção III apresenta um estudo de caso, exemplificando o uso da plataforma CORBA em ambientes heterogêneos. Por fim, as seções IV e V apresentam o estado da arte e as conclusões, respectivamente.

II. CORBA

CORBA é um padrão criado pela OMG (*Object Management Group*) para especificar uma forma simples de comunicação entre aplicações de sistemas distribuídos. Criando uma camada de abstração que permite a invocação de métodos remotamente com facilidade e interoperabilidade de linguagens de programação e arquiteturas de hardware.

A. Contexto Histórico

Em 1989 foi criado a OMG, um consórcio no qual participavam 11 empresas, entre elas estão a IBM, HP, Sun Microsystems e Apple Computer. O objetivo desse grupo era definir especificações de padrões para o gerenciamento de objetos distribuídos e sua semântica de invocação. Atualmente estão focados em padrões de modelagem de sistemas. Em 1991 foi apresentado o CORBA 1.0 que especificava um modelo de objeto, IDL (*Interface Definition Language*) e um conjunto básico de APIs (*Application Programming Interfaces*), porém essa versão inicial só suportava a linguagem C, ou seja, ainda não permitia interoperabilidade entre outras linguagens de programação. Nas duas versões seguintes, 1.1 e 1.2, foram corrigidas algumas especificações ambíguas da primeira versão. Em 1996, a versão 2.0 foi apresentada com diversas melhorias relacionadas à interoperabilidade, como o mapeamento para C++. Seguido, em 1998, a versão 2.2 com mapeamento para Java, assim, CORBA começou a ser mais utilizado. As versões seguintes trouxeram revisões das especificações e melhorias para aplicações de tempo real, sendo que, a última versão, a 3.0 foi apresentada em 2002 [8].

B. Object Management Architecture

A OMA (*Object Management Architecture*) propõe uma arquitetura básica e define um modelo de referência para sistemas distribuídos baseados em objetos. Em sua concepção, não foi intenção da OMG criar um modelo flexível e abrangente, mas sim um modelo específico a OMG, que tem como objetivo a implementação de um guia de referência para

a tecnologia existente. Como um modelo, especifica os requisitos conceituais que qualquer sistema deve possuir para estar de acordo com o padrão.

O desenvolvimento dessa arquitetura seguiu certos objetivos técnicos de modo a agregar benefícios ao gerenciamento de objetos. A seguir são listados alguns desses objetivos [13]:

- **Conformidade:** deve-se manter a compatibilidade na herança de interfaces, onde as interfaces devem possuir pelo menos a interface da superclasse, e também deve manter compatibilidade na herança de implementação, onde a chamada a um método derivada implica na execução do código da classe pai.
- **Transparência de distribuição:** deve ser transparente para o programador todos os aspectos referentes a distribuição de objetos tais como: localização, forma de acesso, mecanismos de comunicação, meios de armazenamento, segurança e tipos de máquinas e de sistemas operacionais.
- **Desempenho para operações remotas e locais:** deve apresentar bom desempenho e escalabilidade mesmo com o aumento do número de clientes de um certo objeto, seja ele local ou remoto.
- **Comportamento extensível e dinâmico:** adição de novas funcionalidades e alterações em métodos não deveria alterar outros objetos ou exigir sua alteração.
- **Arquitetura com serviço de nomes:** oferecer um forma única de acesso a um objeto (através de seu nome), e também permitir a criação de contextos para os nomes.
- **Pesquisas baseadas em nomes, atributos e relacionamentos:** é importante que além da busca por nome do objeto, também seja possível a pesquisa do objeto por outras formas, para o caso do nome do objeto não ser conhecido, por exemplo, uma pesquisa baseada nos atributos ou serviços que eles provêm.
- **Controle de acesso:** controlar quais objetos remotos em um servidor estarão disponíveis aos clientes. É possível utilizar políticas restringindo o acesso a certos clientes.
- **Controle de concorrência:** assegurar que a consistência de um objeto não seja comprometida por invocações concorrentes.
- **Transações:** algumas atividades, mesmo que formadas por operações diferentes, precisam ser atômicas e, dessa forma, o controle de versões se faz necessário.
- **Operações robustas e de alta disponibilidade:** o sistema deve recuperar-se em caso de falha e informar os clientes caso necessário.
- **Notificação de eventos:** oferecer um mecanismo de notificação de eventos remotos, permitindo que processos distribuídos possam ser notificados sobre a ocorrência de um evento em outro processo.

A OMG descreveu na OMA um modelo mais abstrato, ou seja, de mais alto nível, que atende aos objetivos técnicos descritos acima. CORBA é a concretização dessa descrição genérica. Pode-se dizer que a OMA serve com "tipo" e o

CORBA uma "instância" desse tipo. A arquitetura e os elementos que fazem parte da OMA são vistos na Figura 1.

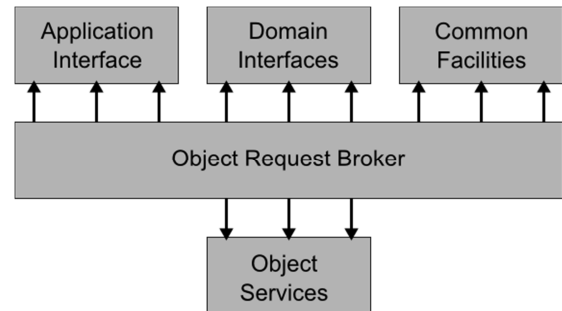


Figura 1 - Visão Geral da Arquitetura OMA [15]

- **ORB (Object Request Broker):** é responsável por prover o meio de comunicação entre os objetos de maneira transparente.
- **Services:** são serviços de propósito geral que estendem as funcionalidades do ORB e que são relevantes para a operação básica de uma aplicação distribuída. Ex.: serviços de nomes, de eventos, de segurança, etc.
- **Domains:** são denominados serviços verticais, soluções parciais para aplicações de domínio específico ligado às áreas de mercado, como interfaces de domínio em telecomunicação.
- **Facilities:** são as interfaces ou serviços, denominados de horizontais, que podem ser utilizados por vários Domains.
- **Applications:** representa a grande variedade de aplicações desenvolvidas como, por exemplo, interfaces comerciais proprietárias.

C. Componentes do padrão CORBA

O objetivo das seções a seguir é apresentar os elementos que constituem o padrão CORBA.

1) Object Request Broker

O núcleo da arquitetura especificada no OMA é o ORB (*Object Request Broker*), o qual é definido pelo padrão CORBA. É o componente de comunicação responsável por localizar e inicializar objetos, assim como mapear a comunicação entre clientes e servidores, fornecendo uma forma única de acesso aos serviços. Objetos utilizam o ORB para comunicação com outros objetos em um ambiente distribuído.

O ORB é, portanto, um *middleware* de comunicação que permite que os clientes façam requisições e recebam respostas de objetos servidores, os quais podem estar localizados localmente ou remotamente em relação ao cliente. Muitos autores, pelo fato do ORB gerenciar essa comunicação, o denominam como sendo o barramento de objetos [15].

Para garantir que uma invocação a um método remoto seja executada, mesmo no caso de falhas intermitentes na rede, o ORB define diferentes semânticas de invocação.

A invocação de métodos remotos no CORBA tem como padrão a invocação “no máximo uma vez”. Nesta semântica, uma requisição é retransmitida até que haja uma confirmação de recebimento e execução no objeto remoto. O objeto remoto,

- *Basic Object Adapter* (BOA)
- *Portable Object Adapter* (POA)

O BOA foi o primeiro adaptador de objeto especificado pela OMG, mas a sua especificação foi duramente criticada. Devido a especificação imprecisa do BOA, problemas relativos a portabilidade de aplicativos existentes para outras ORBs ocorreram. Complicações adicionais surgiram porque a funcionalidade especificada não cobria todo o potencial das aplicações, de modo que cada fornecedor disponibilizou sua própria funcionalidade adicional.

Com a especificação do POA, a OMG forneceu definições mais precisas sobre *client*, *server*, *CORBA object*, *servant*, *object identity* e *object reference*. Em suma, estes termos podem ser caracterizados da seguinte forma:

- **Client:** Um contexto computacional que invoca (remoto) operações em um objeto e, para isso, acessa uma referência de objeto para aquele objeto.
- **Server:** Um contexto computacional no qual a implementação de um objeto CORBA está.
- **CORBA Object:** Um objeto CORBA é uma entidade virtual fornecida com uma identidade, uma interface e a implementação correspondente.
- **Servant:** O *servant* é uma entidade que existe no contexto de um servidor e implementa a interface IDL para um ou mais objetos CORBA. O *servant* é responsável pelo tratamento dos pedidos de um cliente.
- **Object Identity:** No contexto do POA, a identidade do objeto é usada para identificar um determinado objeto CORBA. Os IDs dos objetos são normalmente

gerenciados pelo POA.

- **Object Reference:** A referência do objeto é a ligação entre clientes e servidores. Ele contém toda a informação necessária para invocar uma operação em um objeto remoto. A referência do objeto encapsula os detalhes da localização do objeto, como por exemplo o endereço IP (*Internet Protocol*), o número da porta, o ID do objeto, além de um infinidade de detalhes adicionais.

c) *Repositório de Interface*

O Repositório de Interfaces ou IR, do inglês *Interface Repository*, é responsável por armazenar as definições de interface de objetos CORBA. Os metadados armazenados no IR podem ser acessados em tempo de execução, a fim de gerar invocações dinâmicas usando a *Dynamic Invocation Interface* ou a *Dynamic Skeleton Interface*. O IR é basicamente um servidor CORBA, que deve ser acessado separadamente a partir dos processos de cliente e servidor e pode estar localizado em qualquer lugar na rede. O IR é descrito por um conjunto de interfaces IDL que determinam como o IR pode ser acessado remotamente.

Existem diferentes formas de acesso ao IR. Independentemente do tipo escolhido, o objetivo comum a todos é determinar como um objeto CORBA é estruturado. Por exemplo, se um cliente obteve uma referência de objeto para um objeto CORBA, mas não tem nenhuma informação sobre as operações ou atributos suportados por esse objeto, ele pode adquirir essa informação em tempo de execução. Chamando a respectiva operação, o cliente obtém a descrição da interface pertencente a referência passada pela chamada. Subsequentemente, o cliente pode identificar quais argumentos devem ser passados, quais os tipos presentes, e se

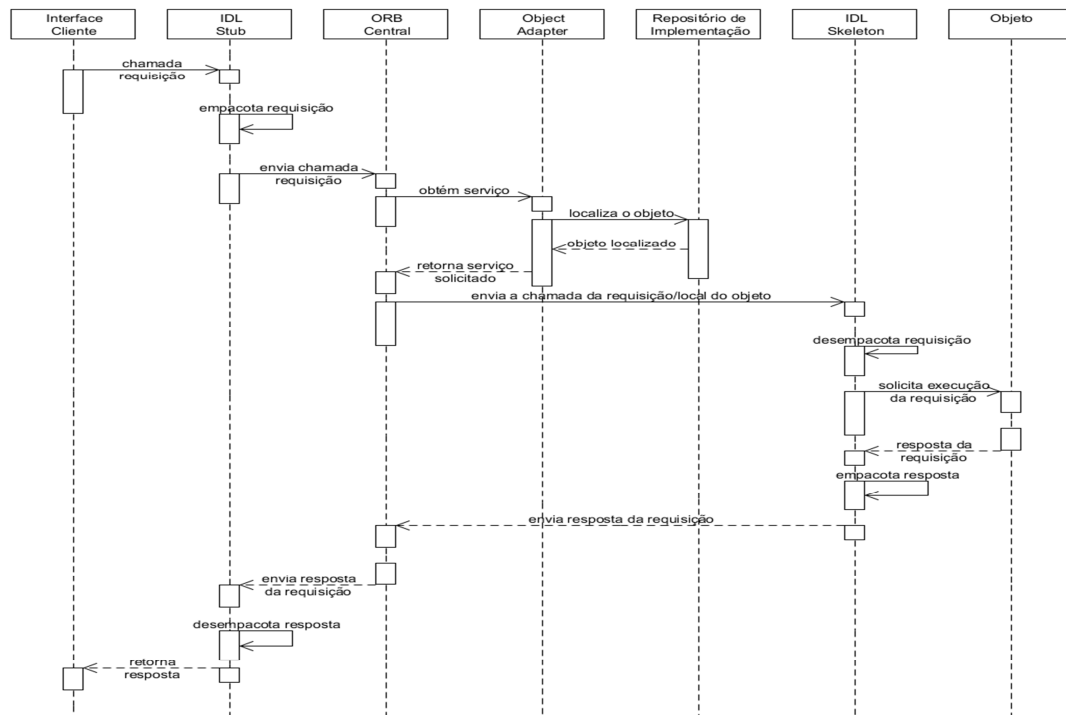


Figura 3 - Visão geral de requisições estáticas no CORBA [16]

há valor de retorno à chamada. Posteriormente, o cliente pode fazer uso da interface de invocação dinâmica, a fim de enviar uma solicitação para o objeto.

Normalmente o RI é um componente independente e nem todos os produtos ORB disponíveis fornecem esse componente. De qualquer modo, para muitas aplicações de CORBA uma implementação IR não é necessária.

d) Repositório de Implementação

Um dos componentes do CORBA mencionado no padrão, mas que não é especificado em detalhe, é o Repositório de Implementação. Sua tarefa é administrar diferentes implementações do servidor. Como a OMG não especificou formalmente o Repositório de Implementação, fica a cargo do desenvolvedor decidir se será implementado e de que forma será sua implementação. Esta é uma desvantagem para o desenvolvimento de aplicações inter-operáveis já que os desenvolvedores não podem assumir que um repositório de implementação está realmente disponível.

e) Stubs e Skeletons

O *stub* é um objeto local através do qual o cliente faz requisições. Pela perspectiva do cliente, o *stub* atende a uma chamada de método localmente, servindo como um intermediário para um objeto remoto. O compilador IDL gera todo o código necessário para esse *stub*, de forma que o programador não tenha que interferir.

O código gerado é, portanto, uma implementação completa de um objeto na linguagem para a qual o compilador IDL possui mapeamento. Este *stub* inclui os procedimentos necessários para a codificação/decodificação dos parâmetros dos métodos, decodificação dos valores de retorno e localização do código remoto através da referência ao *skeleton* do servidor.

O *skeleton* é a implementação equivalente ao *stub* no lado

do servidor. Assim como o *stub*, ele também é gerado automaticamente pelo compilador de IDL. Ele contém o código responsável pela decodificação dos parâmetros dos métodos e codificação dos valores de retorno para a linguagem suportada.

Stubs e *skeletons* utilizam uma representação de dados única, garantindo a compatibilidade de dados entre diferentes linguagens de programação, plataformas de software ou hardware. Nesta representação são oferecidos 15 tipos primitivos, os quais incluem *short*, *long*, *unsigned short*, *float* e *double*, além de tipos compostos (estruturas e enumerações).

A Figura 5, adaptada de [17], exemplifica a representação de um *struct Person*, com os valores: { 'Smith', 'London', 1934 }. Esta ilustração descreve 4 bytes por linha, sendo que cada caractere ocupa 1 byte. Os dados de comprimento variável, como os strings, tem espaços preenchidos com “_” nesta representação simplificada.

Sequência de bytes ← 4 bytes →		
0-3	5	Tamanho do string
4-7	"Smit"	'Smith'
8-11	"h "	
12-15	6	Tamanho do string
16-19	"Lond"	'London'
20-23	"on "	
24-27	1934	unsigned long

Figura 5 – Representação comum de dados

Os processos de empacotamento (*marshalling*) e desempacotamento (*unmarshalling*) de dados utilizam esta abordagem. Portanto, com base na IDL, *stub* e *skeleton* são

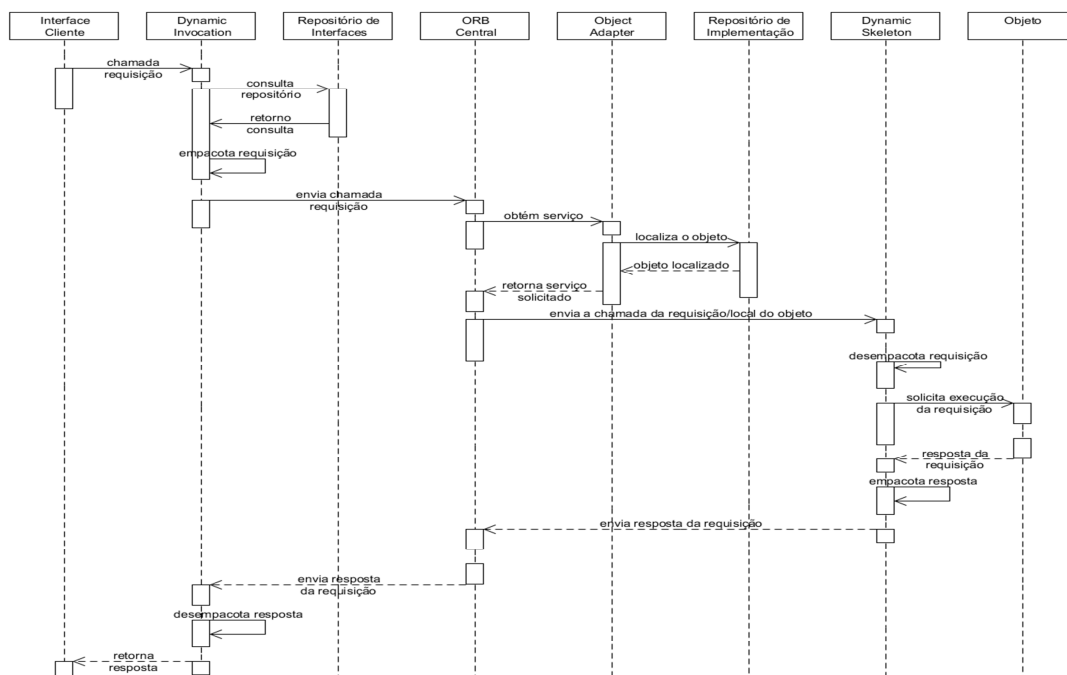


Figura 4 - Visão geral de requisições dinâmicas no CORBA[16]

capazes de interpretar corretamente os dados recebidos e atribuí-los corretamente aos seus respectivos tipos de dados.

Finalmente, o código que implementa a interface descrita pela IDL no lado do servidor é o *servant*. O *servant* é criado explicitamente pelo programador e deve conter a implementação dos métodos do objeto remoto.

f) *Invocação estática de métodos*

Assim que uma definição de interface é compilada por um compilador IDL, são gerados dois elementos fundamentais: o *stub* do lado do cliente e o *skeleton* de implementação no lado do servidor. Esses elementos fornecem a interface estática para acesso aos serviços.

A Figura 3 apresenta um diagrama de sequência com uma visão geral da chamada e retorno de requisições estáticas. Basicamente, o cliente executa a chamada da requisição por meio da interface da implementação do objeto. O IDL *stub* empacota a requisição que é enviada por meio do ORB. O ORB utiliza como o auxílio para localizar o objeto os serviços do *Object Adapter* e do repositório de implementação. Após ser localizado o objeto ocorre a execução e em seguida a resposta da requisição é enviada para o cliente.

g) *Invocação dinâmica de métodos*

No método de invocação estática, para que um objeto (cliente) faça uma requisição para uma implementação (servidor) é necessário o prévio conhecimento da interface, ou seja, das assinaturas de cada um dos métodos da implementação.

Para muitas aplicações esta abordagem é suficiente. Entretanto, no momento em que se necessita de uma maior flexibilidade, outro mecanismo se faz necessário. Assim, a invocação dinâmica de métodos do CORBA tem como objetivo permitir que, em tempo de execução, objetos sejam descobertos bem como seus métodos e seus parâmetros. Assim, através da DII (*Dynamic Invocation Interface*) um cliente pode alcançar qualquer objeto em tempo de execução e invocar seus métodos sem a necessidade de *stubs* pré-compilados.

A Figura 4 apresenta o diagrama de sequência da chamada e retorno de requisições dinâmica. O fluxo é bastante semelhante ao método estático, mas no método dinâmico o DII faz uma consulta ao repositório de interfaces para descoberta do objeto.

Os clientes podem descobrir implementações de várias formas. Uma delas é realizar uma consulta pelo nome do objeto ao *Naming Service*, o qual pode fornecer uma referência ao objeto. Outra forma é procurar por objetos no *Trader Service*. Após a fase de descoberta do objeto é necessária a criação de um objeto intermediário, o pseudo-objeto CORBA *Request*, que define as operações sobre um objeto remoto, como por exemplo, as operações *get_response*, *invoke*, *send_onway* e *delete*.

2) CORBA Services

CORBA Services são coleções de objetos (serviços) que acrescentam e complementam o ORB com funcionalidades básicas úteis para diversas aplicações. As interfaces desses serviços são especificadas em IDL, seguindo os padrões do CORBA.

Esses serviços são como qualquer outro objeto CORBA, sem nenhuma relação especial com o ORB, mas apresentam funcionalidades fundamentais aplicáveis a várias aplicações.

Uma das vantagens dos CORBA Services serem como qualquer outro objeto é a de que podem ser implementados por qualquer um. Usuários de um ORB podem obter e utilizar um CORBA Service de outro ORB ou também podem fazer suas próprias implementações no caso da ausência de um serviço. Os serviços descritos pela OMG são brevemente descritos a seguir.

Properties: Permite que se associe valores nomeados (propriedades) com qualquer componente ou estado de um componente.

Relationship: Oferece uma forma de criar associações dinâmicas entre componentes que não se conhecem. Pode ser utilizada para forçar integridade referencial, relação de agregação e outras formas de ligação entre componentes.

Query: Fornece operações de consulta a objetos. É um super conjunto de SQL, baseado no SQL3 e no *Object Database Management Group's* (ODMG) *Object Query Language* (OQL).

Externalization: Oferece meios padronizados de tratar a entrada e saída de dados de um componente através de mecanismos de *stream*.

Persistence: Provê uma interface única para armazenar componentes em diferentes meios de armazenamento, como simples arquivos ou bancos de dados relacionais ou orientados a objetos.

Collection: Fornece interfaces CORBA para criar e manipular as formas mais comuns de coleções.

Concurrency: Oferece meios para bloquear (obter locks) tanto em threads quanto em transações.

Transaction: Oferece mecanismos de transação (inclusive aninhadas) e recuperação.

Naming: Oferece meios para um componente encontrar outro pelo nome. Permite a criação de contextos de nomes.

Life Cycle: Define operações para criar, copiar, mover e remover componentes.

Licensing: Oferece operações para controle de acesso aos componentes. Permite controle de licenças por sessão, por nodo, por instância ou por site.

Trader: Fornece um meio para os objetos publicarem seus serviços e procurarem por serviços registrados.

Security: Ferramenta completa para segurança em objetos distribuídos. Suporta autenticação, lista de controle de acesso, confidencialidade, etc.

Time: Oferece interfaces para sincronização de tempo em ambiente distribuído. Provê também operações para definir e gerenciar eventos controlados (ou disparados) por tempo.

Além dos serviços já mencionados, CORBA oferece o modelo de eventos e notificação remota. Este modelo é o sucessor de tecnologias de *middleware* orientados a

mensagens, como por exemplo, o JMS e o modelo *publish/subscribe* [19].

a) Serviço de Eventos

A especificação do serviço de eventos CORBA define um modelo onde alguns objetos são capazes de gerar eventos (fornecedores) e outros de consumir eventos (consumidores). As notificações de evento são geradas pelo fornecedor, mas podem ser propagadas aos consumidores de duas maneiras: (i) o fornecedor envia notificações aos objetos interessados no evento (*push*) ou; (ii) os consumidores buscam notificações nos fornecedores (*pull*).

Independente do modo de propagação do evento, objetos remotos são utilizados para estabelecer a referência entre consumidores e fornecedores. A notificação do evento em si, é passada como um parâmetro (no estilo *push*) ou resultado (no estilo *pull*) do tipo *any*.

Para permitir que eventos sejam notificados de modo assíncrono, são utilizados canais de evento. Estes canais são objetos CORBA que possibilitam que vários fornecedores possam gerar eventos e vários consumidores possam consumi-los. Para assinar um canal, um objeto obtém, através do serviço de nomes, referência a algum objeto *proxy* deste canal. De forma análoga, fornecedores também registram-se aos objetos *proxy* do canal, onde são passadas as notificações de eventos. A Figura 6 ilustra 3 objetos compondo o canal de eventos e 2 objetos, fornecedor e consumidor, escrevendo e lendo do canal, respectivamente.

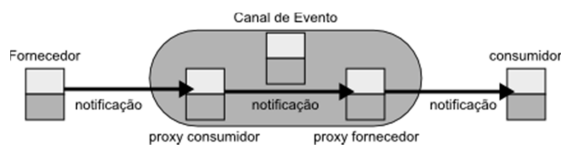


Figura 6 – Canal de eventos

b) Serviço de Notificação

O serviço de notificação é uma extensão do serviço de eventos. Neste serviço, as notificações de evento não ficam restritas ao tipo *any*. É possível utilizar estruturas para representar tipos de dados representando as notificações.

Além disso, os consumidores podem especificar um conjunto de eventos de interesse utilizando filtros. Dessa forma, nem todos os eventos gerados por um fornecedor, são propagados para seus consumidores. Algumas propriedades foram adicionadas aos canais de evento buscando maior confiabilidade na transmissão dos eventos. Por exemplo, é possível estipular prioridade na propagação de eventos, garantia de ordenação na entrega (ordem FIFO) e definir uma política de descarte de eventos no canal.

3) CORBA Facilities e CORBA Domains

Os CORBA Services, vistos na seção anterior, são as interfaces fundamentais padronizadas pela OMG. Essas interfaces podem ser utilizadas para a construção de aplicações complexas, possuindo papel semelhante às chamadas de sistema de um sistema operacional. As aplicações, por sua vez, são as tecnologias especializadas que não são padronizadas pela OMG.

CORBA Facilities, que também são chamadas de Horizontal Facilities, representam, portanto, capacidades de mais alto nível as quais buscam a interoperabilidade das aplicações oferecendo serviços mais complexos em áreas como gerência da interface com o usuário, gerência de informações, gerência do sistema e gerência de tarefas.

Por sua vez, CORBA Domains são as áreas verticais do mercado, por exemplo: serviços financeiros, de saúde, de manufatura, telecomunicações e objetos de negócios. Ao contrário dos CORBA Facilities, CORBA Domains não buscam a interoperabilidade entre os domínios, mas apenas dentro do seu próprio domínio, assumindo, dessa forma, uma visão mais vertical.

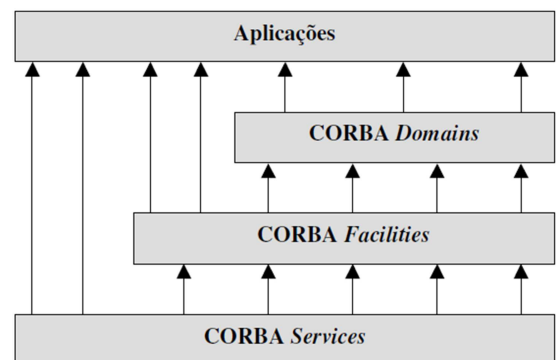


Figura 7 - Níveis de dependência do CORBA [16]

Assim, a última categoria de objetos, os quais não são padronizados pela OMG, os objetos de aplicação, podem utilizar as características fornecidas tanto pelos CORBA Services como pelos CORBA Facilities e CORBA Domains. Isto visa uma maior produtividade e desenvolvimento mais rápido e seguro através da utilização de objetos já desenvolvidos para um conjunto de atividades que se mostraram necessárias para as mais variadas aplicações. A Figura apresenta a relação de dependência entre esses níveis da arquitetura CORBA.

III. ESTUDO DE CASO

Esta seção ilustra o potencial de CORBA ou integrar aplicações desenvolvidas em diferentes plataformas. Os exemplos apresentados foram extraídos de [13], que utiliza a plataforma CORBA TAO para código em C++ e IIOP.NET para código C#. Algumas situações comuns em sistemas corporativos heterogêneos são descritas pelo autor, como segue:

A. Clientes heterogêneos: Implementação em C++ e C#

Este cenário ilustra situações em que novas tecnologias são adotadas por seguirem tendências de mercado ou por prover melhor usabilidade ou conveniência para os usuários. Portanto, objetos remotos devem estar acessíveis tanto para as novas aplicações, quanto para as aplicações legadas ainda em uso.

A Figura 8 descreve este cenário, onde o servidor foi implementado em C++, assim como as aplicações cliente legadas. Em adição, foram criadas novas aplicações cliente em C#.

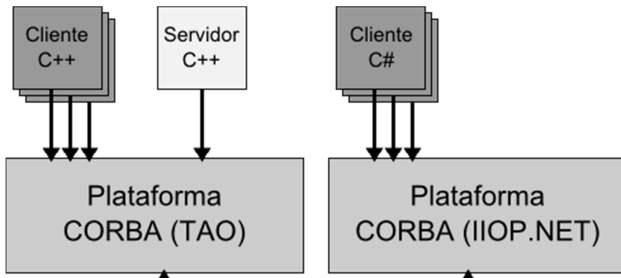


Figura 8 - Níveis de dependência do CORBA

Por simplicidade, o exemplo apresentado implementa apenas um método *add*, que soma dois números *double* e retorna o valor da soma. Este método é implementado por um objeto remoto mantido pelo servidor. A IDL para a criação do servidor e cliente C++ é descrita a seguir:

```
1. module ExampleInterfaces{
2.     interface IAdder{
3.         double add(in double arg1,
4.                   in double arg2);
5.     };
6. }
```

A compilação da IDL para o TAO é feita através do comando: `tao_idl -GI ExampleInterfaces.idl` e como resultado da compilação são gerados os códigos para o *stub*, *skeleton* e, devido ao uso da diretiva de compilação `-GI`, também será gerado o código base para o *servant*. Este arquivo deverá conter a implementação dos construtores, inicializações e métodos do objeto remoto. A seguir é mostrado o trecho de código referente ao método *add*:

```
1. CORBA::DoubleExampleInterfaces_IAdder_i::
   add(::CORBA::Double arg1, ::CORBA::Double
   arg2)ACE_THROW_SPEC((CORBA::SystemExcepti
   on)){
2.     return (arg1 + arg2); // Implementação
3. }
```

A seguir é apresentada parte da implementação do servidor. Para melhor visualização, foram separados apenas trechos de código ilustrando a interação da aplicação C++ com a plataforma CORBA TAO. Os tratamentos de exceção foram omitidos do código apresentado.

1) *Importação das bibliotecas da plataforma CORBA e biblioteca gerada pelo compilador de IDL:*

```
1. #include "ExampleInterfaces_i.h"
2. #include<orbsvcs/CosNamingC.h>
3. #include<orbsvcs/Naming/Naming_Client.h>
4. #include<ace/streams.h>
```

2) *Cria e inicializa uma instância ORB:*

```
1. CORBA::ORB_var orb = CORBA::ORB_init(
   argc, argv );
```

3) *Obtém referência da raiz POA:*

```
1. CORBA::Object_var obj = orb->
   resolve_initial_references("RootPOA");
2. PortableServer::POA_var poa =
   PortableServer::POA::_narrow(obj.in());
```

4) *Ativação do gerenciador POA:*

```
1. PortableServer::POAManager_var mgr = poa->
   the_POAManager();
2. mgr->activate();
```

5) *Busca CORBA Naming Service:*

```
1. CORBA::Object_var naming_obj = orb->
   resolve_initial_references("NameService")
   ;
2. CosNaming::NamingContext_var root =
   CosNaming::NamingContext::_narrow(naming_
   obj.in());
```

6) *Associa o NamingContext a um nome conhecido pelos clientes*

```
1. CosNaming::Name name;
2. name.length(1);
3. name[0].id =
   CORBA::string_dup("ExampleInterfaces");
4. CORBA::Object_var dummy = root->
   resolve(name);
```

7) *Cria um servant e o registra no Naming Service*

```
1. ExampleInterfaces_IAdder_i adderServant;
2. PortableServer::ObjectId_var oid = poa->
   activate_object(&adderServant);
3. CORBA::Object_var adderServant_obj = poa->
   id_to_reference(oid.in());
4. root->rebind(name, adderServant_obj.in());
```

8) *Aceita requisições de objetos clientes*

```
1. orb->run();
2. orb->destroy();
```

De forma análoga, a seguir são apresentados trechos do código cliente escrito em C++, destacando principalmente as interações da aplicação com a plataforma CORBA.

1) *Importação das bibliotecas da plataforma CORBA e biblioteca gerada pelo compilador de IDL:*

```
1. #include "ExampleInterfacesC.h"
2. #include<orbsvcs/CosNamingC.h>
3. #include<orbsvcs/Naming/Naming_Client.h>
4. #include<ace/streams.h>
```

2) *Cria e inicializa um ORB*

```
1. CORBA::ORB_var orb = CORBA::ORB_init(
    argc, argv );
```

3) *Obtém uma referência ao contexto de nomeação de raiz*

```
1. CORBA::Object_var naming_obj = orb->
    resolve_initial_references("NameService")
    ;
2. CosNaming::NamingContext_var root =
    CosNaming::NamingContext::_narrow(naming_
    obj.in());
```

4) *Resolve o objeto baseado no nome*

```
1. CosNaming::Name name;
2. name.length(2);
3. name[0].id =
    CORBA::string_dup("ExampleInterfaces");
4. name[1].id=CORBA::string_dup("IAdder");
5. CORBA::Object_varobj = root->
    resolve(name);
```

5) *Atribui valores as variáveis e invoca método remoto*

```
1. cout << "Enter the first number: ";
2. cin >> number1;
3. cout << "Enter the second number: ";
4. cin >> number2;
5. sum = iAdder->add(number1, number2);
6. cout << "The sum is: " << sum << endl;
```

Para a criação de clientes C#, não está sendo utilizada uma plataforma CORBA completa, mas apenas a implementação de um canal de comunicação compatível com o protocolo IIOP sobre o *framework .NET Remoting* [18]. Portanto, diferentemente do cliente em C++, que executa sobre a plataforma TAO, a aplicação C# não utiliza uma definição de linguagem IDL. O código a seguir exibe a interface C# utilizada.

```
1. /// IAdder.cs
2. using System;
3.
4. namespaceSLB.ExampleInterfaces{
5.     /// Interface IAdder aceita dois
6.     /// doubles e retorna um double
7.     public interface IAdder{
8.         double add(double arg1, double arg2);
9.     }
10. }
```

Partes do código C# da aplicação cliente são apresentadas a seguir:

1) *Importação das bibliotecas da plataforma CORBA e interface do objeto:*

```
1. usingSystem.Runtime.Remoting;
2. usingSystem.Runtime.Remoting.Channels;
3. usingCh.Elca.Iiop;
4. usingCh.Elca.Iiop.Services;
5. usingomg.org.CosNaming;
6. usingSLB.ExampleInterfaces;
```

2) *Criação e registro do Canal*

```
1. IiopClientChannel channel = New
    IiopClientChannel();
2. ChannelServices.RegisterChannel(channel);
```

3) *Acesso ao serviço de nomes*

```
1. CorbaInit init = CorbaInit.GetInit();
2. NamingContext nameService =
    init.GetNameService(host, port);
```

4) *Acesso o módulo que define a IDL mapeada para namespace .NET*

```
1. NameComponent[] moduleName = new
    NameComponent[]{
2.     new NameComponent("ExampleInterfaces",
        "")};
3. NamingContextnameSpace =
    (NamingContext)nameService.resolve(modu
    leName);
```

5) *Acesso o módulo que define a IDL mapeada para classe .NET*

```
1. NameComponent[] interfaceName = new
    NameComponent[]{new NameComponent(
    "IAdder", "")};
2. SLB.ExampleInterfaces.IAdderservant =
    (SLB.ExampleInterfaces.IAdder)nameSpace.r
    esolve(interfaceName);
```

6) *Utilização do servant*

```
1. InteractWithUser(servant);
```

B. *Substituição do servidor, sem alterar o funcionamento dos clientes*

Este segundo exemplo ilustra a atualização da aplicação no servidor para uma nova tecnologia. Esta situação é bastante comum e é desejável que não haja incompatibilidade com aplicações clientes em uso. A Figura 9 ilustra a substituição do servidor C++ por uma nova versão implementada em C#.

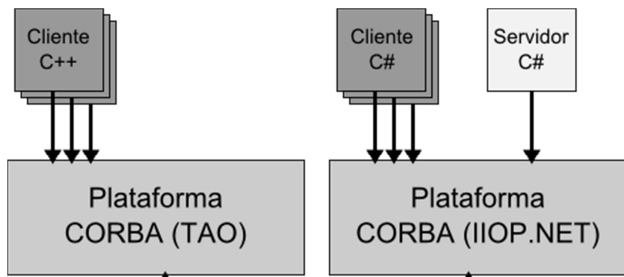


Figura 9 - Níveis de dependência do CORBA

Os códigos para aplicações cliente C++ e C# não sofrem alterações. A seguir será descritos trechos de código do servidor implementado em C#.

1) *Implementação da interface IAdder, ou seja, a implementação do objeto a ser disponibilizado:*

```
1. public double add(double arg1, double
   arg2){
2.     return arg1 + arg2;
3. }
```

2) *Para a publicação da implementação e registro no serviço de nomes do TAO, utilizando o protocolo IIOP, inicia-se registrando o canal.*

```
1. IiopChannel chan = new IiopChannel(0);
2. ChannelServices.RegisterChannel(chan);
```

3) *Acesso ao Naming Service do CORBA*

```
1. CorbaInitinit = CorbaInit.GetInit();
2. NamingContextnameService =
   init.GetNameService(host, port);
```

4) *Associação com o módulo de definição da interface mapeada para o namespace .NET*

```
1. NameComponent[] moduleName =
   newNameComponent[]{newNameComponent("Exa
   mpleInterfaces", "")};
2. NamingContextnameSpace =
   nameService.bind_new_context(moduleName);
```

5) *Associação com o módulo de definição da interface mapeada para a classe .NET que implementa o objeto.*

```
1. NameComponent[] interfaceName =
   newNameComponent[]{
   newNameComponent("IAdder", "")};
2. Adder interfaceImplimentation = new
   Adder();
3. nameSpace.bind(interfaceName,
   interfaceImplimentation);
```

A partir da execução de *bind*, o objeto é associado ao serviço de nomes CORBA e já pode ser invocado remotamente pelos clientes.

IV. ESTADO DA ARTE

Assim como o CORBA, outras tecnologias também implementam objetos distribuídos com o intuito de oferecer um modelo de programação mais familiar a maioria dos programadores. Criando a abstração de objetos distribuídos sobre primitivas de chamadas a procedimentos remotos (RPC - *Remote Procedure Call*), a Microsoft estende a tecnologia COM (*Component Object Modelo*) para sua versão distribuída. DCOM (*Distributed Component Object Model*) [3] define uma camada de abstração sobre as camadas do DCE RPC [2]. A Sun Microsystems também propõe um padrão para uso de objetos remotos na linguagem Java, chamado Java RMI (*Remote Method Invocation*) [7]. Destacam-se, mais recentemente, as Arquiteturas Orientadas a Serviços (SOA - *Service Oriented Architectures*), que se baseiam em objetos distribuídos para implementar o modelo de serviços Web. Este paradigma oferece transparência ao programador e facilidade na integração entre sistemas [1] [12].

CORBA e DCOM utilizam o modelo cliente/servidor para comunicar objetos distribuídos. No entanto, a instanciação de objetos (locais ou remotos) e chamadas a métodos esconde do programador o gerenciamento de conexões e trocas de mensagens entre nodos distribuídos. Outras características do paradigma orientado a objetos também são oferecidas por estes modelos de objetos distribuídos, como o encapsulamento de dados, polimorfismo e herança. Entretanto, CORBA também implementa o conceito de herança múltipla e especificação de exceções, indisponíveis na especificação DCOM.

Diferentemente de CORBA e DCOM, Java RMI não permite comunicação entre linguagens de programação diferente. Implementado como parte da máquina virtual Java, RMI utiliza a serialização de objetos oferecida pela linguagem para codificar parâmetros e referências a objetos durante invocação de métodos remotos. Com relação à definição de interface, Java RMI oferece maior transparência ao programador, pois utiliza sintaxe Java para definição de tipos de atributos e classes, sem depender de uma IDL específica. Embora o desenvolvimento de aplicações Java seja favorecido pela uniformidade de tipos oferecida, a grande vantagem de CORBA sobre Java RMI ainda é a independência de plataforma e linguagem de programação.

Mais detalhes sobre cada uma destas especificações podem ser encontradas em [11]. Os autores traçam um comparativo entre CORBA, DCOM e RMI relacionando aspectos como: protocolos de requisição e recebimento, referência remota, descrição de interface (IDL), codificação de dados, nomeação e registro (repositório) de objetos.

SOA apresenta um modelo de serviços fracamente acoplado e bastante familiar a modelos de negócio. A facilidade com que serviços são oferecidos a clientes amplamente distribuídos e possibilidade de reuso são algumas das características deste modelo. O Protocolo SOAP, responsável por trocar mensagens em SOA, faz uso de protocolos comuns para o desenvolvimento Web, como o HTTP e XML. Uma das grandes vantagens do SOAP sobre CORBA, é o transporte através do protocolo HTTP. Isso garante que o tráfego gerado por chamadas a serviços Web não seja barrado por *firewalls*, evitando configurações

adicionais de infraestrutura. Além de simples, protocolos usados por serviços Web são de fácil aprendizado para a maioria dos programadores e este paradigma é totalmente independente de plataforma.

Apesar da popularidade de SOA e serviços Web para a integração entre sistemas corporativos, o protocolo SOAP apresenta algumas limitações, como a falta de modelo de segurança de dados e a falta de canais confiáveis para comunicação entre redes conectadas [1]. Além disso, a representação de dados em modo textual, através de XML, gera maior custo computacional no processamento de requisições SOAP. Em [4] os autores comparam DCOM e SOAP, identificando uma constante perda de desempenho dos serviços Web, quando comparados ao modelo de objetos distribuídos.

Uma vantagem de CORBA sobre outros modelos de objetos distribuídos é o fato deste propor uma especificação aberta, possibilitando diferentes implementações deste *middleware*. Atualmente, encontram-se disponíveis implementações de CORBA integradas às linguagens de programação C, C++, Java, Smalltalk, Ruby, Python, Lisp, Cobol, entre outras [8]. Além disso, aprimoramentos e adição de funcionalidades à especificação são constantes. Por exemplo, na versão 3.0 da especificação CORBA, estão disponíveis recursos para tolerância a falhas [9] e execução de tarefas de tempo real [10].

Por se tratar de um padrão aberto, iniciado no início dos anos 90, a especificação CORBA evoluiu muito, sendo bastante genérica, porém extensa. Ainda que apresente diversas vantagens, como heterogeneidade, tolerância a falhas e implementações de ORBs estáveis, a maior dificuldade em utilizar CORBA reside na configuração da plataforma. Em [5] os autores analisam dois ORB confiáveis, que implementam a especificação FT-CORBA: o OGS e o Eternal. Segundo os autores, ambas plataformas oferecem recursos bastante genéricos, permitindo, por exemplo diversas configurações para técnicas de replicação. No entanto, o excesso de alternativas de configuração torna a preparação dos ambientes bastante dispendiosa e passível de erros.

V. CONCLUSÃO

CORBA assumiu um importante papel no desenvolvimento de *middleware* baseados em objetos distribuídos no início dos anos 90, porém ainda aparece como uma importante alternativa para desenvolvimento de sistemas distribuídos especializados (embarcados, tolerantes a falhas ou de tempo real). Devido ao seu padrão aberto, além de possibilitar implementações para diferentes linguagens de programação e plataformas, aprimoramento e adição de funcionalidades são constantemente incorporadas à sua especificação.

Com a popularização dos sistemas distribuídos e diversidade de tecnologias existentes, CORBA foi uma das primeiras arquiteturas para construção de sistemas distribuídos heterogêneos, destacando-se como excelente alternativa para integração entre sistemas. Conforme apresentado na Seção IV, propostas mais modernas, como SOA e serviços Web, tornaram-se bastante populares para a integração de sistemas distribuídos. Uma das grandes vantagens de SOA sobre

CORBA está no fato deste utilizar protocolos da Internet e XML. Portanto, ao invés de utilizar uma camada de *middleware* como plataforma de comunicação, SOA possibilita a criação de requisições a serviços utilizando protocolos HTTP para transporte e codificação de dados baseada em XML.

Embora a utilização de serviços web tenha superado a utilização de CORBA na construção de sistemas corporativos, ainda há casos em que a arquitetura CORBA é preferível. Diferentemente de serviços web, os objetos distribuídos mantêm seu estado persistente e são referenciados por nodos clientes. Portanto, aplicações que necessitam compartilhar conteúdo executam operações de leitura e escrita de atributos sobre em objetos (distribuídos ou locais) carregados em memória. Além de proporcionar melhor desempenho, por não necessitar de um mecanismo externo para persistência (como um servidor de banco de dados), neste modelo não há qualquer restrição arquitetural sobre sua infraestrutura. Portanto, diferente de SOA, não há necessidade em utilizar servidores de aplicação ou servidores de banco de dados.

Com relação ao desempenho, sistemas de monitoramento remoto são um exemplo de aplicação onde objetos remotos são mais adequados que serviços web. Simples objetos residentes em memória coletam métricas de utilização de recursos, possibilitando que monitores leiam seus atributos periodicamente sem gerar custo computacional elevado. Sistemas autônomos e adaptativos muitas vezes não dispõem de acesso a infraestruturas fixas. Nestes casos, CORBA é capaz de oferecer alto nível de abstração, independência de plataforma e ainda oferecer serviços especializados para tolerância a falhas e execução de tarefas em tempo real [1].

REFERÊNCIAS

- [1] F. Curbera, M. Duftler, R. Khalaf, W. Nagy, N. Mukhi, and S. Weerawarana. Unraveling the Web Services: An Introduction to SOAP, WSDL, and UDDI. *Internet Computing*, IEEE, 6(2):86–93, 2002.
- [2] DCE 1.1: Remote Procedure Call Specification, The Open Group. The Open Group website: <http://www.rdg.opengroup.org/public/pubs/catalog/c706.htm/> (acesso em março/2012).
- [3] DCOM – Distributed Component Object Model. MSDN website: <http://msdn.microsoft.com/en-us/library/aa286561.aspx> (acesso em março/2012).
- [4] A. Davis and D. Zhang. A Comparative Study of DCOM and SOAP. *Fourth International Symposium on Multimedia Software Engineering*, 2002.
- [5] P. Felber and P. Narasimhan. Experiences, Strategies, and Challenges in Building Fault-Tolerant CORBA Systems. *IEEE Transactions on Computers*, 53:497–511, 2004.
- [6] M. Henning. The Rise and Fall of CORBA. *Queue*, 4(5):28–34, 2006.
- [7] Java Microsystems: Java Remote Method Invocation Specification. Sun Microsystems website: <http://java.sun.com/j2se/1.5/pdf/rmi-spec-1.5.0.pdf> (acesso em março/2012).
- [8] OMG – Object Management Group. CORBA website: <http://www.corba.org/> (acesso em março/2012).
- [9] OMG Doc: formal/2010-05-07. OMG, Fault Tolerance CORBA spec. v1.0. The Open Group website: <http://www.omg.org/spec/ft/1.0> (acesso em março/2012).
- [10] OMG Doc: formal/2005-01-04. OMG, Real-Time CORBA spec.v1.2. The Open Group website: <http://www.omg.org/spec/rt/1.2> (acesso em março/2012).
- [11] F. Plásil and M. Stal. An Architectural View of a Distributed Objects and Components in CORBA, Java RMI and COM/DCOM. *Software Concepts and Tools*, 19:14–28, 1998.

- [12] J. Roy and A. Ramanujan. Understanding Web Services. IT Professional, 3(6):69 –73, 2001.
- [13] S. Bogner. Mixing ACE/TAO and .NET Clients and Servers. The Code Project website: <http://www.codeproject.com/articles/12275/mixing-ace-taoand-net-clients-and-servers> (acesso em março/2012).
- [14] P. Alan. The CORBA Reference Guide: Understanding the Common Object Request Broker Architecture. USA: Addison- Wesley, 1998.
- [15] C. Alexandre. Uma Abordagem de Escalonamento Adaptativo no Ambiente Real-Time CORBA. Dissertação de Mestrado 2002.
- [16] S. Ana Paula G. O Modelo de Arquitetura CORBA e suas Aplicações. 2004.
- [17] C. Jean Dollimore and Tim Kindberg. Distributed Systems: Concepts and Design (4th Edition) (International Computer Science). Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA. 2005.
- [18] Microsoft. MSDN Library - .NET Remoting. <http://www.microsoft.com/germany/msdn/library/net/NETRemoting.aspx>
- [19] P. Th. E., P. A. Felber, R. Guerraoui, and A. Kermarrec. 2003. The Many Faces of Publish/Subscribe. ACM Comput. Surv. 35, 2003), 114-131.